

SOMATIVA

Pokédex



Nome: Henrique Rodrigues Motta

Turma: 3 DEVT-B

Professor: Bruno

Matéria: PBE - Programação Back-End

Sobre o Projeto

Este projeto foi desenvolvido utilizando o framework PHP Laravel com o objetivo de criar uma API e um sistema de cadastro de Pokémons inspirado na Pokédex.

O sistema permite:

- Buscar Pokémons através da PokeAPI;
- Cadastrar novos Pokémons;
- Fazer upload de imagens e áudios;
- Armazenar dados no banco de dados;
- Trabalhar com rotas API utilizando JSON.

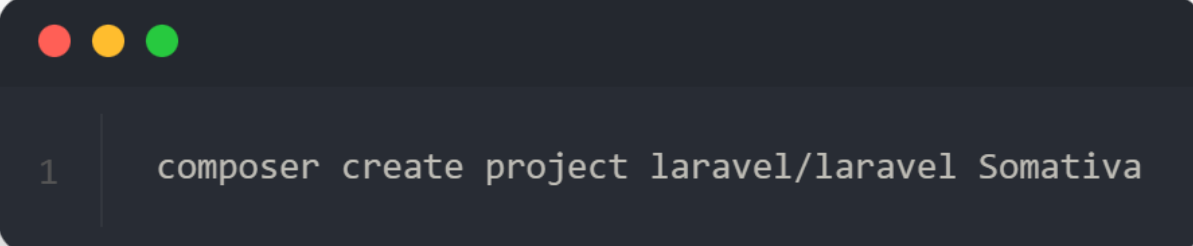
Tecnologias Utilizadas

- PHP
- Laravel
- MySQL
- Composer
- Artisan
- API Externa (PokeAPI)

Configurações Iniciais do Projeto

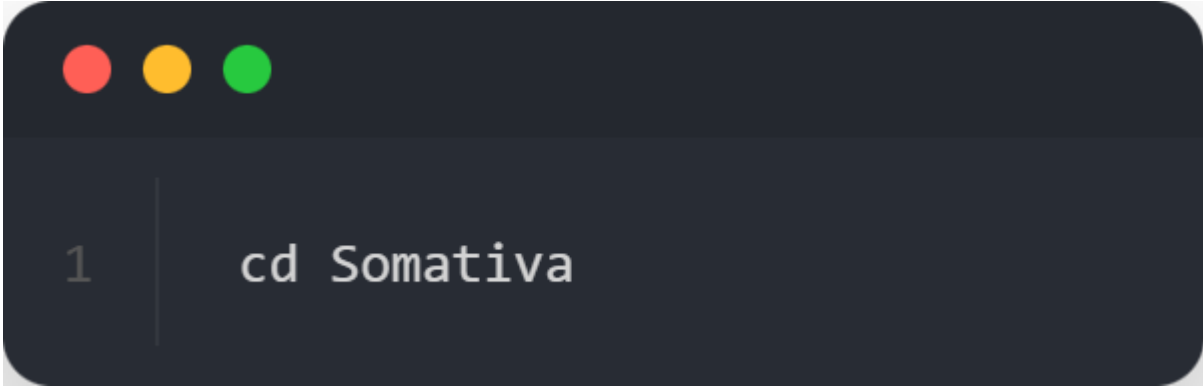
1. Criação do Projeto Laravel

O projeto foi criado utilizando o Composer através do comando:



```
1 | composer create-project laravel/laravel Somativa
```

Após a instalação, foi acessada a pasta do projeto:

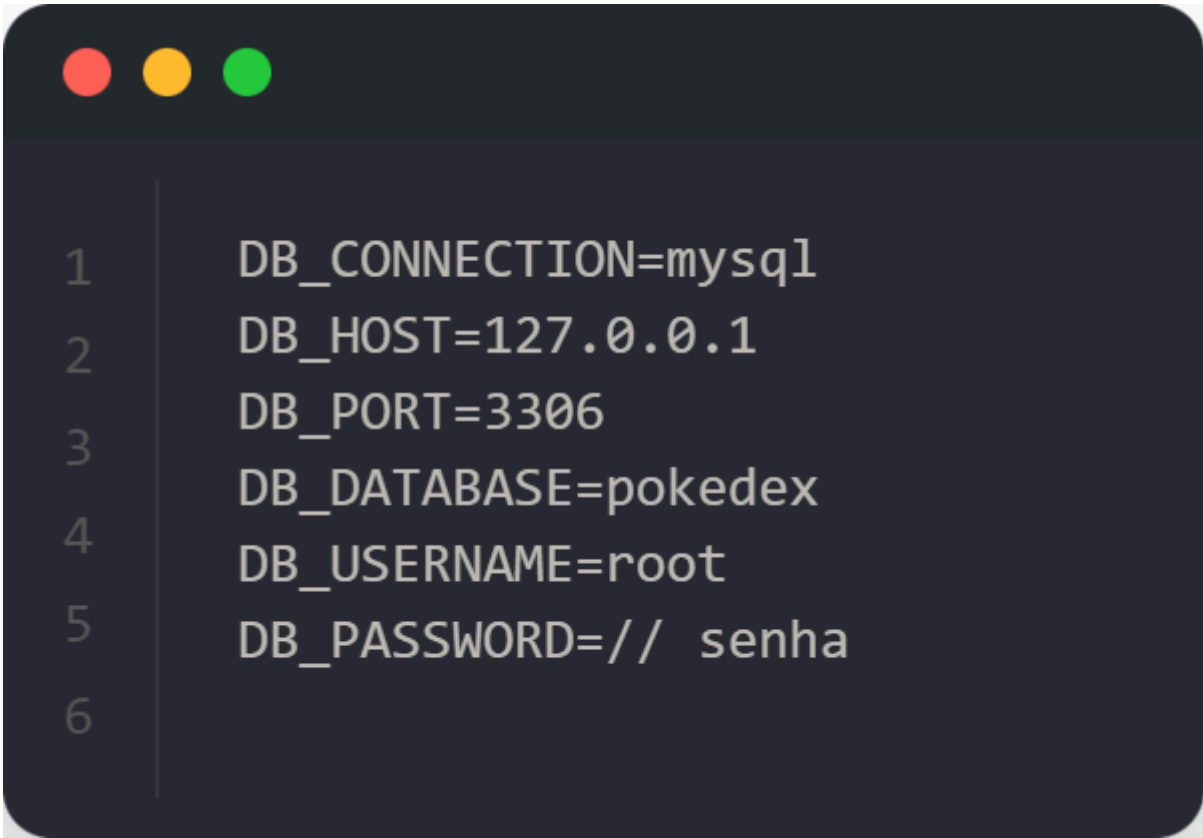
A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top. The command 'cd Somativa' is entered on line 1.

```
1 cd Somativa
```

Configuração do Arquivo .env

O arquivo `.env` foi utilizado para configurar a conexão com o banco de dados.

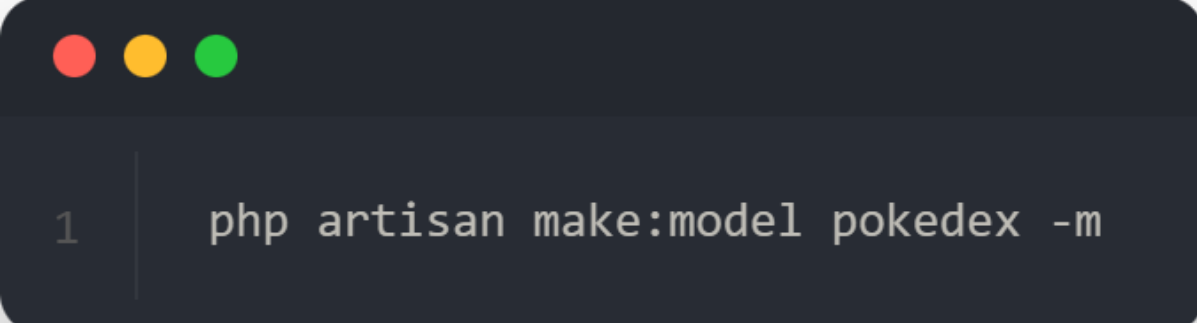
Exemplo:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top. The contents of the .env file are displayed on lines 1 through 6.

```
1 DB_CONNECTION=mysql
2 DB_HOST=127.0.0.1
3 DB_PORT=3306
4 DB_DATABASE=pokedex
5 DB_USERNAME=root
6 DB_PASSWORD=// senha
```

Criação do Banco de Dados

Primeiramente foi criado o banco de dados, e sua migration e o model com o seguinte comando:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. A light gray line indicates the cursor position. The command 'php artisan make:model pokedex -m' is entered, with a '1' in the left margin.

```
1 | php artisan make:model pokedex -m
```

Depois disso, o Laravel foi configurado para acessar o banco utilizando os dados definidos no `.env`.

Migrations

As migrations foram utilizadas para gerar automaticamente as tabelas do banco de dados. A tabela do banco de dados é a seguinte:

```
1 public function up(): void
2 {
3     Schema::create('pokedexes', function (Blueprint $table) {
4         $table->id();
5         $table->string('POKE_name', 100)->unique();
6         $table->json('POKE_elements');
7         $table->float('POKE_height');
8         $table->float('POKE_weight');
9         $table->json('POKE_stats');
10        $table->string('POKE_generation');
11        $table->string('POKE_sprite')->nullable();
12        $table->string('POKE_shiny')->default(false);
13        $table->json('POKE_abilities');
14        $table->string('POKE_audio')->nullable();
15        $table->integer('POKE_xp');
16        $table->timestamps();
17    });
18    DB::statement('ALTER TABLE pokedexes AUTO_INCREMENT = 1');
19 }
20
21 public function down(): void
22 {
23     Schema::dropIfExists('pokedexes');
24 }
25
```

Após criar as migrations, foi executado o comando:

```
1 php artisan migrate
```

Esse comando criou todas as tabelas do sistema no banco de dados.

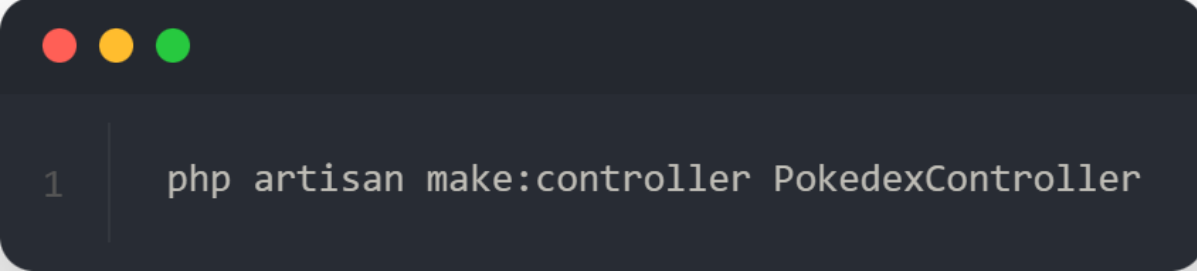
Model

O model é responsável por fazer a comunicação entre o Laravel e o banco de dados. Sendo ele presente abaixo:

```
1  use Illuminate\Database\Eloquent\Model;
2
3  class Pokedex extends Model
4  {
5      // Corrigido 'POKE_habilities' para 'POKE_abilities' e alterado para protected
6      protected $fillable = [
7          "POKE_name",
8          "POKE_elements",
9          "POKE_height",
10         "POKE_weight",
11         "POKE_stats",
12         "POKE_generation",
13         "POKE_sprite",
14         "POKE_shiny",
15         "POKE_abilities",
16         "POKE_audio",
17         "POKE_xp"
18     ];
19
20     // Adicionado os Casts (Obrigatório para colunas JSON no banco)
21     protected $casts = [
22         'POKE_elements' => 'array',
23         'POKE_stats' => 'array',
24         'POKE_abilities' => 'array',
25     ];
26 }
27
28
```

Criação do Controller

O controller foi criado utilizando:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. A single line of code is entered in the terminal, preceded by a line number '1' in a light gray font.

```
1  php artisan make:controller PokedexController
```

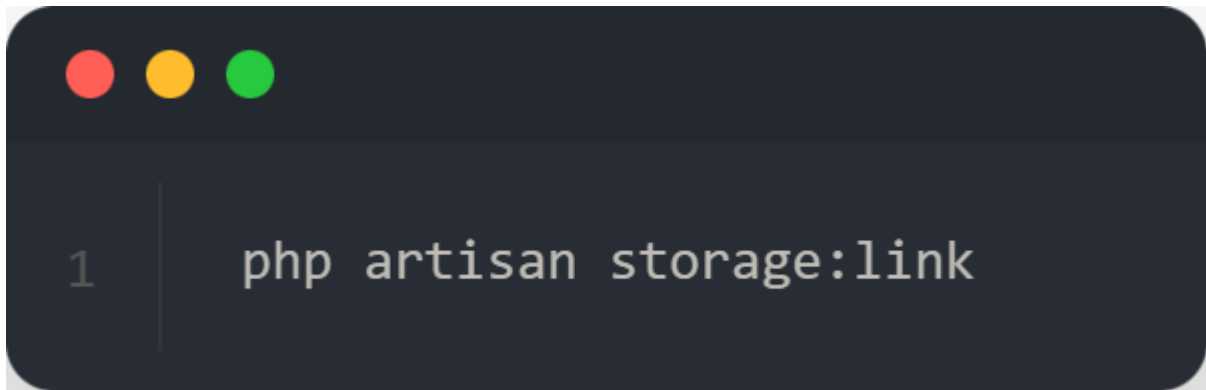
O controller é responsável por controlar as requisições do sistema.

Exemplo de funções implementadas:

- Cadastro de Pokémons;
- Validação de dados;
- Upload de imagens;
- Upload de áudio;
- Retorno JSON.

Configuração do Storage

Para permitir o acesso aos arquivos enviados para o sistema (imagens e áudios), foi utilizado o comando:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The command 'php artisan storage:link' is entered on a single line, preceded by a line number '1' and a vertical cursor line.

```
1 | php artisan storage:link
```

Esse comando cria um link simbólico entre:

`storage/app/public`

E:

`public/storage`

Com isso, os arquivos enviados podem ser acessados pelo navegador.

Rotas da API

Buscar Pokémon pela API Externa

Rota:

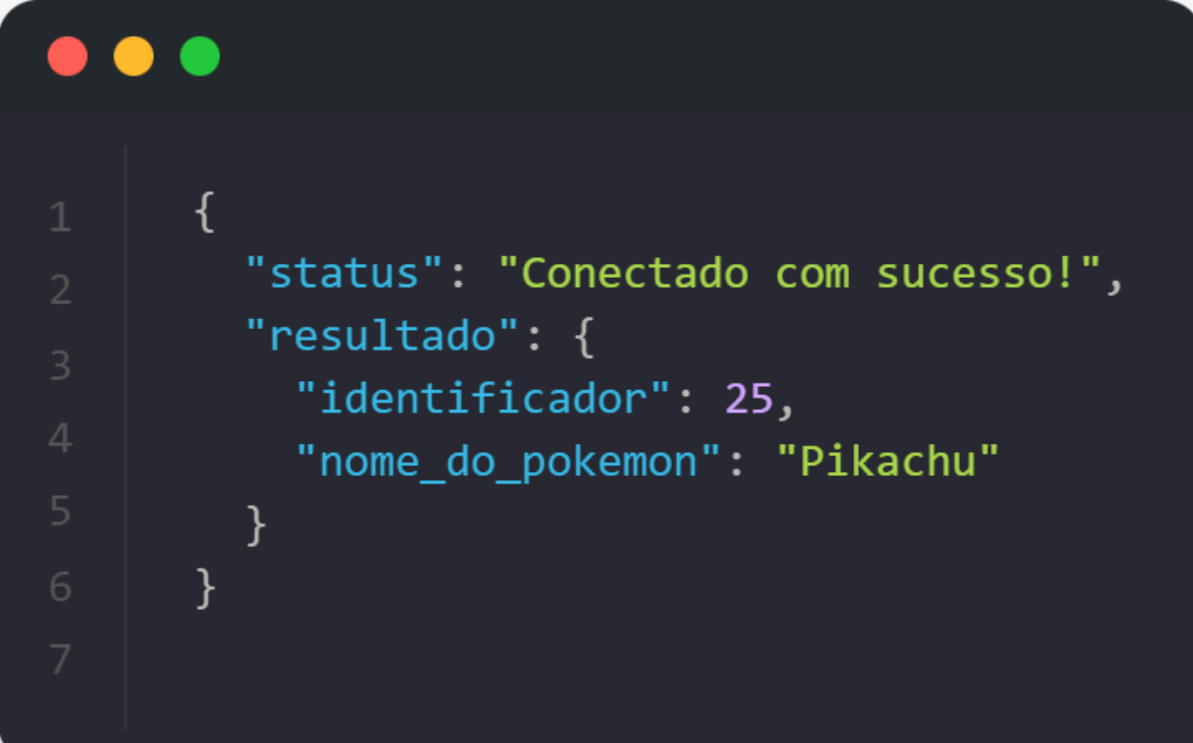
GET /api/pokemon/{nome}

Essa rota faz uma requisição para a PokeAPI utilizando o facade `Http` do Laravel.

Exemplo:

GET /api/pokemon/pikachu

Retorno esperado:



```
1  {
2    "status": "Conectado com sucesso!",
3    "resultado": {
4      "identificador": 25,
5      "nome_do_pokemon": "Pikachu"
6    }
7  }
```

Upload de Arquivos

O sistema permite upload de:

- Sprite do Pokémon;
- Sprite Shiny;
- Áudio do Pokémon.

Os arquivos são armazenados utilizando:

```
store('pokemons', 'public')
```

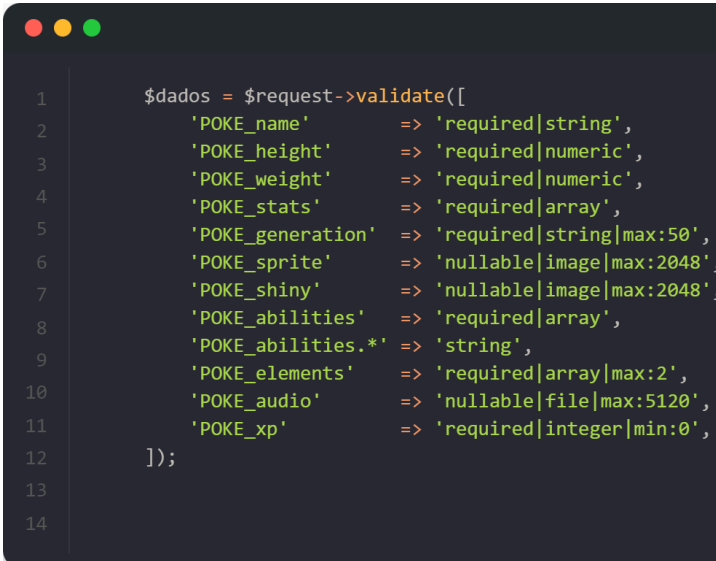
E:

```
store('cries', 'public')
```

Validação de Dados

Foi utilizado o método `validate()` do Laravel para validar os dados recebidos.

Exemplo:



```
1      $dados = $request->validate([
2          'POKE_name'      => 'required|string',
3          'POKE_height'    => 'required|numeric',
4          'POKE_weight'    => 'required|numeric',
5          'POKE_stats'     => 'required|array',
6          'POKE_generation' => 'required|string|max:50',
7          'POKE_sprite'    => 'nullable|image|max:2048',
8          'POKE_shiny'     => 'nullable|image|max:2048',
9          'POKE_abilities' => 'required|array',
10         'POKE_abilities.*' => 'string',
11         'POKE_elements'   => 'required|array|max:2',
12         'POKE_audio'      => 'nullable|file|max:5120',
13         'POKE_xp'         => 'required|integer|min:0',
14     ]);
```

Isso garante que os dados enviados estejam corretos antes de salvar no banco.

Estrutura do Projeto

Principais Pastas

app/

Contém os controllers e models do sistema.

database/

Contém migrations, seeders e configurações relacionadas ao banco.

routes/

Contém as rotas web e API.

storage/

Contém os arquivos enviados pelos usuários.

public/

Pasta pública acessada pelo navegador.

Considerações Finais

Durante o desenvolvimento do projeto foram aplicados conceitos importantes do Laravel, como:

- Rotas API;
- Controllers;
- Models;
- Migrations;
- Upload de arquivos;
- Validação de formulários;
- Integração com APIs externas;
- Organização MVC.

O projeto serviu como prática para desenvolvimento backend utilizando Laravel e integração com banco de dados.